

Skriva data till fil

Att skriva data till en fil är betydligt mycket enklare än att läsa data. Man kan se på det som att skriva till skärmen, men data skrivs istället till en fil.

Den klass som används för att skriva data är `ofstream`. Även denna finns definierad i `<stream>`. För att skriva till en fil kan vi t.ex. använda följande syntax:

```
#include <fstream>
#include <stdlib.h>

int main () {
    // öppna filen
    ofstream Ut ( "index.html");

    // kunde filen öppnas?
    if ( ! Ut ) {
        // kunde inte öppna filen!
        cout << "Kunde inte öppna filen 'index.html'!" << endl;
        exit ( EXIT_FAILURE );
    }

    // skriv en html-version av 'Hello world'
    Ut << "<html><head>" << endl;
    Ut << "<title>Hello world</title>" << endl;
    Ut << "<body>" << endl;
    Ut << "Hello world" << endl;
    Ut << "</body></html>" << endl;
}
```

Filen vi får ser ut på följande sätt:

```
% ./Writel
% cat index.html
<html><head>
<title>Hello world</title>
<body>
Hello world
</body></html>
```

Alla normala datatyper kan skrivas ut, helt på normalt sätt.

Det finns en del flaggor som kan användas då man öppnar filer för skrivning, precis som när man öppnar dem för läsning. Dessa flaggor finns definierade i `ios` och är:

- `app` öppnar filen för *appendering*, d.v.s. data skrivs till på slutet av filen, istället för som normalt skriva över en fil. Kan användas för t.ex. logfiler.
- `ate` öppnar filen och "söker" till slutet av filen.
- `binary` öppnar filen i binär form. Ingen inverkan under Unix.

- `out` öppnar en fil för skrivning. Standard för `ofstream`.
- `trunc` öppnar en fil för skrivning och *trunkerar* innehållet, d.v.s. skriver över allt innehåll. Standard för `ofstream`.

De vanligaste flaggorna man använder är `ios::app` för att skriva till slutet av en fil.

Skriva teckenvis

Precis som man kan läsa teckenvis kan man även skriva data teckenvis. Man kan då använda metoden `put()` för att skriva ut ett enda tecken. Denna används i följande program för att skriva ut alla programmets kommandoradsparametrar till en fil `params.txt`

```
#include <fstream>
#include <stdlib.h>
#include <string>

int main (int argc, char * argv[]) {
    // öppna filen
    ofstream Ut ( "params.txt" );

    // kunde filen öppnas?
    if ( ! Ut ) {
        // kunde inte öppna filen!
        cout << "Kunde inte öppna filen 'params.txt!' << endl;
        exit ( EXIT_FAILURE );
    }

    // iterera över alla parametrar vi har
    for ( int Index = 0; Index < argc; Index++ ) {
        // hämta parameter
        string Parameter = argv[Index];

        // iterera över alla tecken i strängen
        for ( unsigned int Tecken = 0; Tecken < Parameter.length (); Tecken++ ) {
            // skriv ut ett tecken
            Ut.put ( Parameter[Tecken] );
        }

        // skriv ett radbyte
        Ut.put ( '\n' );
    }
}
```

Vi indexerar här strängen `Parameter` tecken för tecken och skriver ut dem. Vi kunde lika väl ha direkt skrivit ut strängen med hjälp av `<<`.

Skriva binär data

Man kan läsa binär data i C++, så följdaktligen måste det finnas ett sätt att även skriva binär data. Man kan använda sig av metoden `read()` för att läsa binärt, så för att skriva används en metod som heter `write()`. Denna metod tar som argument en pekare och ett tal som anger antalet *bytes* som skall skrivas ut. Man kan här använda `sizeof()` ifall man inte är säker. Vi kan nu skriva ett program som skriver ut ett antal `int`:s i binär form. Denna data kan läsas av programmet i [avsnittet Läs binär data](#).

```
#include <fstream>
```

```
#include <stdlib.h>

int main () {
    // öppna filen
    ofstream Out ( "Test.bin" );

    // kunde filen öppnas?
    if ( ! Out ) {
        // kunde inte öppna filen!
        cout << "Kunde inte öppna filen 'Test.bin'!" << endl;
        exit ( EXIT_FAILURE );
    }

    // skriv ut 10 tal till filen
    for ( int Index = 0; Index < 10; Index++ ) {
        // skriv ut talet binärt
        Out.write ( &Index, sizeof(int) );
    }
}
```

Notera att vi inte använder flaggan `ios::binary` i detta exempel, eftersom flaggan inte har någon verkan under Unix. Under Windows (och andra system) kanske flaggan är nödvändig.

Som tidigare nämnts (se [avsnittet Läs binär data](#)) bör man inte använda binära filer om inte omständigheterna kräver det.

[Förutgående](#)

Läs från en fil

[Hem](#)

[Upp](#)

[Nästa](#)

Hantera felsituationer